

“Hospital MediFlow Tracker Dashboard”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

I. Harshika : 2520030607

B. Bhavitha :

2520030120

Under the Esteemed Guidance of

Dr K. Madhavi

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Declaration

The Project Report entitled **“Design and Development of an Integrated Tourism Platform for Flights, Trains, Hotels, and Hidden Destinations”** is a record of Bonafide work of **I.Harshika - 2520030607** , **B.Bhavitha - 2520030120**, submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

I.Harshika – 2520030607

B.Bhavitha – 2520030120

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled **“Design and Development of an Integrated Tourism Platform for Flights, Trains, Hotels, and Hidden Destinations”** is a bonafide work done and submitted by **I.Harshika-2520030607, B.Bhavitha-2520030120,** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2025-2026.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr.K.Madhavi** , and my guide **Dr.K.Madhavi**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the ***Department for Computer Science Engineering/ Department for Computer Science and Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

I.Harshika – 2520030607

B.Bhavitha– 2520030120

ABSTRACT

In healthcare facilities, inefficient bed management and poor patient flow tracking often lead to delays in treatment and reduced quality of care. The Hospital MediFlow Tracker Dashboard is designed to solve these challenges by providing a real-time, centralized platform for monitoring hospital operations.

The platform is structured around six primary, fully interconnected modules. It offers real-time bed availability tracking across wards (available, occupied, or under maintenance), complete patient admission and discharge management, and treatment tracking. Crucially, the platform includes a comprehensive Alert and Notification System for emergency alerts, medicine reminders, and staff notifications, addressing the high demand for real-time communication in healthcare environments.

The most important feature is the dedicated Patient Journey Tracking Module. This system tracks every stage of a patient's hospital stay from admission to discharge, including medications, doctor visits, treatment progress, and inter-ward transfers. By promoting data-driven hospital operations, we are not only improving patient outcomes but also supporting hospital staff coordination and enabling faster emergency response. The system is built using robust, modern web technologies and is fully responsive across all devices. In summary, this project delivers a complete, reliable, and user-friendly digital solution that significantly enhances the efficiency of managing hospital beds, patients, and operations in real time.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-15
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	16-21
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	22-32
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	33-36
5	Design	5.2 ER Diagram / Database schema	37
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	38-47
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	48-55
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	56-64
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	65-73
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	74-82

11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	83-88
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	89-97
13	References	- Books, tutorials, APIs, documentation sites used	98-99
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	100-105

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

1. Healthcare management is one of the most critical and dynamic sectors globally, directly impacting human lives, national public health infrastructure, and the overall efficiency of medical institutions. The continual advancement of information and communication technologies has transformed how hospitals plan, manage, and monitor their operations. In earlier decades, hospital staff depended heavily on manual registers, paper-based patient records, and verbal coordination to manage ward operations. Today, nearly every aspect of hospital management — from bed allocation and patient admission to treatment tracking and discharge — can be handled through digital platforms and web-based dashboards.
2. Despite this progress, the current hospital management landscape in many institutions remains fragmented. Numerous isolated systems exist for managing patient data, bed availability, and staff communication, yet these services rarely interact with one another in real time. A single hospital often has to switch between several systems, re-enter the same patient information multiple times, and manually coordinate bed assignments across wards. This disjointed process wastes time, increases the risk of administrative errors, and creates significant operational burden on healthcare staff.
3. The idea for this project emerged from observing these inefficiencies and recognizing the opportunity to simplify hospital operations through a unified digital system. The proposed web-based dashboard, "Hospital MediFlow Tracker," aims to provide a single-window solution that consolidates the major components of hospital management into one cohesive platform. By integrating bed management, patient tracking,

treatment monitoring, and alert systems, the dashboard offers hospital staff a seamless, intelligent, and responsive operational experience.

4. Technological growth in web development frameworks such as React.js, cloud computing platforms such as Firebase, and open APIs has made such integration achievable and cost-effective. Many hospitals and healthcare institutions now recognize the need for centralized digital systems that allow administrators, doctors, and nurses to access and update patient and bed data through standardized interfaces in real time. By leveraging these technologies, the platform can display synchronized hospital data instantly and give staff the tools to make quick, informed decisions. The website is built using modern responsive-design principles so that it functions smoothly across laptops, tablets, and smartphones.
5. Beyond the technical aspect, the project also addresses an important gap in how patient journeys are monitored within hospitals. Most existing systems highlight only admission data, leaving treatment progress, inter-ward transfers, and discharge summaries poorly tracked. To counter this imbalance, the system introduces a comprehensive patient tracking module that monitors every stage from admission to discharge, including medication schedules, doctor visits, and recovery progress. By drawing attention to these operational details, the project encourages better care coordination and supports clinical staff whose responsibilities span multiple patients across multiple wards.
6. In summary, the background of this project rests on three fundamental observations: existing hospital management systems are disjointed and inconvenient for modern healthcare staff; rapid technological progress now makes full integration of such systems practical and scalable; and healthcare needs digital inclusivity — bringing bed management, patient tracking, alerts, and administrative controls into a single equitable system.

By addressing these points, the project establishes itself as a pioneering step toward an intelligent, comprehensive hospital management ecosystem.

1.2 Problem Statement

In today's healthcare environment, hospital staff face immense operational challenges due to the lack of a unified, real-time system for managing beds, patients, and ward activities. Despite the availability of various digital tools, most hospitals still operate with fragmented processes — nurses manually check bed availability, doctors access patient files from isolated systems, and administrators have no consolidated view of the hospital's current status. This fragmented approach causes delays in patient admission, errors in bed assignments, poor communication between departments, and ultimately a reduction in the quality of patient care.

The core problem addressed by this project is the absence of an integrated, user-friendly system that combines all major hospital operational components — bed availability, patient admission, treatment tracking, inter-ward transfers, discharge management, and real-time alerts — into one seamless digital environment. Current tools typically focus on a single function and fail to provide holistic support for the entire patient journey. For example, after admitting a patient, staff must still search separately for bed availability in the target ward, update treatment records manually, and communicate discharge plans verbally to other departments. This multi-step, disconnected process causes operational overload and increases the risk of errors and patient dissatisfaction.

Another major issue lies in the inequality of information visibility. Real-time data on bed status, patient conditions, and ward occupancy is not readily accessible to all relevant staff. Administrators, doctors, and nurses often work with stale or

incomplete data, leading to poor decision-making. The absence of a unified notification and alert system further compounds this problem, as critical updates — such as a patient requiring emergency care or a bed becoming available — are often communicated verbally or via manual logs, which are prone to delay and error.

In summary, the problem statement identifies a clear gap between hospital staff's need for a unified, intelligent management dashboard and the current fragmented operational ecosystem. Despite the presence of multiple specialized applications, none provide a fully integrated, data-synchronized, and user-centered approach that addresses the complete patient care cycle. The proposed solution aims to fill this void by delivering a robust, responsive, and comprehensive system that revolutionizes how hospitals manage beds, patients, and operations

1.3 Scope of the Project

The scope of this project, "Hospital MediFlow Tracker Dashboard," extends across multiple dimensions of hospital operations — technological, functional, administrative, and clinical. The main objective is to establish a comprehensive, web-based solution that consolidates the essential services required by hospital staff into a single digital interface. By integrating bed management, patient tracking, treatment monitoring, alerts, and administrative controls, the project aims to redefine how hospitals manage their day-to-day operations in the digital age.

The technical scope involves building a fully responsive, database-driven dashboard that functions efficiently across all devices, including desktops, tablets, and smartphones. The platform is designed using modern web technologies such as HTML, CSS, JavaScript, React.js, Node.js, and Firebase/MongoDB, ensuring scalability, performance, and secure data handling. The system architecture

supports modular expansion, meaning future features — such as appointment scheduling, pharmacy management, or billing integration — can be added without restructuring the entire framework.

In addition, the project encompasses the implementation of real-time data listeners to fetch live bed status and patient data. This ensures that staff receive up-to-date information instead of static listings. The integration of Firebase Real-time Database provides dynamic synchronization between service modules and end-users. The inclusion of a secure authentication and role-based access control ensures smooth and safe data management. User accounts are managed through a centralized authentication system, allowing staff to track patients, manage beds, and receive alerts through personalized dashboards.

The project also falls within the scope of user experience (UX) and interface design, focusing on simplicity, clarity, and accessibility. Hospital staff of all technical backgrounds should be able to navigate the system with minimal effort. The interface is visually clear, using intuitive icons and color-coded status indicators to guide users through the patient and bed management process. Multi-role access can also be considered within the extended scope to make the platform more inclusive for all hospital departments.

- To summarize, the scope of the project can be defined through the following key points:
- Development of a unified, web-based hospital management dashboard.
- Real-time bed and patient tracking with live status updates.
- Integrated alert and notification system for staff communication.
- Scalable system architecture supporting expansion and third-party integration.
- Emphasis on usability, accessibility, and staff satisfaction.

Potential future enhancements through AI-driven analytics and mobile app support.

1.4 Objectives of the Project

The main objective of this project is to design and develop a hospital management dashboard that simplifies operational workflows by unifying all essential components — bed management, patient tracking, treatment monitoring, alerts, and administrative controls — into one dynamic and user-friendly web application.

In the traditional healthcare environment, staff are forced to navigate multiple systems for different aspects of hospital management, such as checking bed availability, updating patient records, or managing alerts. This project's objective is to eliminate such fragmentation by introducing a single-window solution that provides real-time information, tracking tools, and notification functionalities for all major hospital operations within one unified system.

The primary technical objective is to develop a web-based system using modern programming frameworks and responsive design principles that ensure accessibility across all devices. The system should be capable of fetching live data through Firebase real-time listeners, storing patient and bed data securely in a database, and maintaining synchronization among all modules. Each module — Bed Management, Patient Admission, Treatment Tracking, Alerts, and Admin Panel — should function independently but interact seamlessly to deliver a consistent and cohesive staff experience.

To summarize, the major objectives of the project are as follows:

- To design and develop a unified hospital management dashboard integrating all core operational modules.

- To enable real-time data access and updates for bed availability and patient status.
- To provide a responsive, user-friendly interface accessible across all devices.
- To promote efficient patient care through comprehensive journey tracking from admission to discharge.
- To ensure secure and reliable data handling through authentication and encryption mechanisms.
- To offer modular architecture supporting future extensions and scalability.
- To empower hospital administrators and clinical staff with centralized data control.
- To improve emergency response times through an integrated alert and notification system.
- To demonstrate the application of modern web development and software engineering techniques.

1.5 Importance of the Application

The healthcare sector is one of the most critical and time-sensitive domains where operational inefficiency can directly impact human lives. Hospitals that lack real-time visibility into bed occupancy, patient status, and staff assignments face significant challenges in delivering timely and effective care. The Hospital MediFlow Tracker Dashboard addresses these challenges by providing a centralized, real-time solution that transforms how hospitals operate at every level — from ward management to patient discharge.

The proposed system is not just another hospital record system; it is an integrated solution that provides a complete operational experience under one roof. By merging multiple services — bed availability, patient admission and tracking, treatment monitoring, and alert management — the platform offers a single access point for all hospital management needs. This integration saves staff valuable time, reduces complexity, and enhances overall patient satisfaction. For

administrators, doctors, and nurses alike, the application simplifies daily workflows through real-time data synchronization, instant status updates, and intelligent notifications.

One of the most significant aspects of the project's importance lies in its promotion of efficient patient care. The Patient Tracking Module ensures that no stage of a patient's hospital journey is left unmonitored, from initial admission through treatment and finally to discharge. This comprehensive visibility improves clinical decision-making, reduces treatment errors, and ensures timely care for every patient.

- To summarize, the importance of this application can be understood through the following key points:
- It unifies multiple hospital operations — bed management, patient tracking, alerts, and admin controls — into one cohesive platform.
- It saves time, enhances convenience, and reduces staff effort through automation and real-time updates.
- It promotes efficient patient care through complete journey tracking from admission to discharge.
- It empowers hospital administrators and clinical staff through centralized data access.
- It enhances security, reliability, and transparency in patient data management.
- It demonstrates the application of advanced web technologies and system design principles.
- It aligns with national healthcare digitization initiatives promoting smart hospital management.

1.6 Target Users / Audience

The target users of the Hospital MediFlow Tracker Dashboard span a wide and diverse audience across the hospital ecosystem. The platform is designed to serve individuals, departments, and organizations seeking efficiency, coordination, and real-time operational visibility in healthcare management.

The primary users are hospital administrators, doctors, nurses, and receptionists who need to manage beds, track patients, and coordinate care without the hassle of using multiple systems. The platform's single-window interface allows them to access bed availability, patient records, treatment history, and alerts simultaneously, streamlining all key hospital operations.

The secondary users include hospital management and healthcare data analysts who use the platform to monitor performance metrics, generate reports, and ensure compliance with operational standards. By registering on the platform, they gain digital visibility and can track all hospital activities without being present on the floor.

- In essence, the platform caters to:
- Hospital Administrators seeking system-wide operational visibility and control.
- Doctors and medical staff requiring real-time patient status and treatment tracking.
- Nurses responsible for bed assignments, patient care, and ward coordination.
- Receptionists managing patient admissions and discharge processes.
- Hospital management seeking data-driven insights and audit trails for quality assurance.

Overall, the target audience represents a broad network of healthcare professionals who value efficiency, patient safety, and coordinated care. The platform bridges the gap between clinical and administrative functions, offering an ecosystem that benefits all participants in the modern hospital environment.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Hardware requirements form the foundation of any software development process. They define the physical resources necessary to design, develop, test, and deploy the application efficiently. Since this project involves building a full-scale, web-based hospital management dashboard that integrates multiple modules — Bed Management, Patient Tracking, Alerts, and Admin Controls — the underlying hardware must support multitasking, large data processing, and network-based communication in real time. The hardware requirements are divided into two categories: those needed by developers during the design and testing stages, and those needed by end users (hospital staff) to access the deployed system effectively.

The development environment demands moderate to high-end configurations because developers frequently run multiple applications such as integrated development environments (IDEs), database servers, browsers, and local web servers simultaneously. The processor should ideally be an Intel Core i5 or higher or an equivalent AMD Ryzen 5 with at least 2.4 GHz base speed to ensure smooth compilation and server execution. A minimum of 8 GB of RAM is essential, though 16 GB is recommended, especially when running multiple frameworks, virtual machines, or testing environments concurrently.

The server-side hardware requirements depend on the expected user traffic and number of concurrent requests. A moderately configured web server with a quad-core CPU, 16 GB RAM, and 1 TB SSD storage is sufficient for hosting the platform during development or internal testing. For production deployment, using cloud-based infrastructure like Firebase or AWS ensures scalability and automatic load balancing.

From the end user's perspective, the hardware requirements are minimal since the system operates through a web browser. The user's device — whether a desktop, laptop, tablet, or smartphone — needs only a dual-core processor (1.8 GHz or above), 4 GB RAM, and a modern browser capable of running JavaScript-based applications.

Component	Developer Requirement	End User Requirement
Processor	Intel Core i5 / AMD Ryzen 5 (2.4 GHz+)	Dual-core 1.8 GHz or above
RAM	8 GB minimum, 16 GB recommended	4 GB minimum
Storage	250 GB SSD + backup drive	Less than 200 MB (cache/cookies)
Internet	10 Mbps stable broadband	Stable Wi-Fi or mobile data
OS	Windows 10/11, Ubuntu Linux, or macOS	Any OS with a modern browser
Display	Full HD monitor (1920x1080)	Standard smartphone/tablet/PC screen

Browser	Chrome, Firefox, or Microsoft Edge (latest)	Any modern browser with JavaScript
---------	---	------------------------------------

2.2 Software Requirements

Software requirements define the digital environment necessary to design, build, test, and deploy the Hospital MediFlow Tracker Dashboard efficiently. Since the system is developed as a web-based platform, it relies heavily on modern software technologies that enable real-time data exchange, responsive design, and secure data management. A clear understanding of the software stack ensures smooth development and long-term maintainability.

The operating system is a fundamental component of the software environment. For developers, the system can run effectively on Windows 10 or 11, Ubuntu Linux, or macOS. The web server software plays a crucial role in managing user requests and hosting application files. Common options include Apache HTTP Server, Nginx, and Node.js environments.

At the core of the backend, a server-side scripting language handles business logic and communication with the database. This project utilizes Node.js with Express.js for asynchronous handling and faster API communication. Firebase provides authentication, real-time database, and cloud storage services.

- Operating System: Windows 10/11, Ubuntu Linux, or macOS for development
- Frontend Framework: React.js with HTML5, CSS3, and JavaScript (ES6+)
- Styling: Bootstrap 5 / Tailwind CSS for responsive UI
- Backend: Node.js with Express.js for server-side logic
- Database: Firebase (Firestore/Realtime DB) / MongoDB for data storage

- Authentication: Firebase Authentication / JWT-based session management
- Version Control: Git and GitHub for source code management
- IDE: Visual Studio Code as the primary development environment
- Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge for testing
- API Testing: Postman for endpoint validation
- Deployment: Firebase Hosting / AWS EC2 / Netlify for production

Security software is a crucial component of the system's software stack. The platform uses SSL (Secure Socket Layer) certificates to encrypt communication between the client and server. Authentication and authorization can be implemented using JWT (JSON Web Tokens), while bcrypt or similar encryption libraries ensure secure password storage.

2.3 Development Tools and Frameworks

Developing a hospital management dashboard that consolidates multiple real-time services requires a powerful combination of development tools and frameworks. These tools form the backbone of the project's software engineering process, providing structure, efficiency, and maintainability. The use of appropriate tools ensures not only smooth coding but also streamlined testing, version control, and deployment.

At the front-end layer, HTML5 serves as the foundational language for creating the structure of web pages. CSS3 is used for styling and visual enhancement. Through CSS3 features like Flexbox, Grid, transitions, and media queries, the interface becomes adaptive to different devices. For responsiveness and design consistency, Bootstrap 5 or Tailwind CSS is implemented. These frameworks

provide pre-defined classes, utility-first design structures, and cross-browser consistency.

JavaScript (ES6+) forms the core of interactivity on the client side. It enables features such as dynamic content updates, asynchronous communication with Firebase, and form validation without reloading the page. React.js introduces a component-based architecture where the interface is divided into reusable elements such as bed status cards, patient detail panels, ward selectors, and notification banners.

For database management, Firebase Firestore is employed for real-time data synchronization. MongoDB provides flexible document-based storage for patient history logs and audit trails. Database tools help developers manage collections, execute queries, and perform data backups. To ensure real-time data integration, the platform uses Firebase real-time listeners that connect it with live hospital data.

During development, Visual Studio Code (VS Code) serves as the primary Integrated Development Environment (IDE). For version control and collaboration, Git and GitHub are essential tools to track changes, manage branches, and collaborate among developers. Testing forms a major part of the software development cycle. Tools like Postman are used for API testing to verify request-response integrity. Selenium and Jest are used for automated testing of web interfaces and JavaScript functionality respectively.

In terms of deployment and hosting, the project can be deployed using cloud-based services like Firebase Hosting, AWS (Amazon Web Services), or Netlify. Security frameworks play a vital role in protecting user data and system integrity. JWT (JSON Web Tokens) are used for secure session management and user authentication.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The front-end of the Hospital MediFlow Tracker Dashboard represents the user interface layer through which hospital administrators, doctors, nurses, and receptionists interact with the system. This layer is the visual and interactive part of the web application, responsible for delivering a seamless, intuitive, and responsive experience. The front-end architecture has been designed using a combination of HTML5, CSS3, JavaScript, and React.js, ensuring compatibility, scalability, and high performance across all devices.

The core purpose of the front-end design is to allow staff to easily perform all hospital management activities — such as checking bed availability, admitting patients, tracking treatments, and receiving alerts — without any technical difficulty. To achieve this, the platform emphasizes a clean design, minimal navigation layers, and a consistent color theme that aligns with the concept of healthcare and patient safety.

HTML5 forms the backbone of the front-end. It defines the structure of each web page and organizes the content elements like bed status grids, patient cards, ward selectors, alert banners, and admin dashboards. CSS3 is responsible for styling, layout design, and visual presentation. Through features like Flexbox, CSS Grid, and media queries, the design becomes fully responsive. The website's color palette uses calming medical tones to reflect the professionalism required in a healthcare environment.

To streamline front-end development, frameworks such as Bootstrap 5 and Tailwind CSS are incorporated. JavaScript (ES6+) adds interactivity and functionality. It enables the execution of dynamic features such as live bed status updates, real-time patient data, and filtering options for wards and patients. React.js introduces a component-based architecture, where the interface is divided into reusable elements such as BedStatusGrid, PatientCard, WardSelector, AlertBanner, and AdminDashboard. Each component manages its own state and can be updated independently, allowing for rapid UI updates without page reloads.

React also supports React Router, which enables navigation between different pages (e.g., Dashboard, Bed Management, Patient Admission, Treatment Tracking, and Admin Panel) without reloading the site. This Single Page Application (SPA) design provides a desktop-like experience within the browser. Additionally, Axios or Fetch API is used to connect with backend services, sending and receiving data asynchronously in JSON format.

For state management, React Context API or Redux can be implemented. This allows centralized handling of user data such as login sessions, bed assignments, or selected patient preferences. Performance optimization techniques include image compression, code minification, lazy loading of non-critical resources, and caching strategies using browser local storage.

3.2 Back-End Design

The back-end of the Hospital MediFlow Tracker Dashboard serves as the powerful engine that drives all the functionalities and logic of the system, ensuring that user interactions on the front-end are processed efficiently and securely. It is designed using modern technologies that emphasize speed,

scalability, and reliability, with Node.js serving as the primary runtime environment and Express.js as the development framework. Node.js was chosen because of its asynchronous, event-driven architecture that makes it exceptionally capable of handling multiple simultaneous I/O operations — such as fetching real-time bed data, updating patient records, and sending alert notifications — without performance degradation. Express.js, being lightweight and flexible, provides a clean structure for developing RESTful APIs that form the backbone of the application's communication system.

The architecture follows the Model–View–Controller (MVC) pattern, ensuring that logic, data, and interface remain separated, which makes the system modular, easier to maintain, and expandable for future updates. Each incoming request from the user, such as checking available beds or updating a patient's treatment, is routed through Express endpoints where controllers validate the input, interact with the database or Firebase, and then format the data into a JSON response returned to the client.

To ensure secure and smooth communication, the platform uses JSON Web Tokens (JWT) for authentication. When a user logs in, a unique token is generated and attached to every request, ensuring that only authorized staff can access sensitive features like patient management or admin controls. User passwords are encrypted using bcrypt, which converts plain text into a secure hash before storage. Security is further reinforced through middleware like Helmet for setting secure HTTP headers, cors for handling cross-origin requests from the front-end, and express-validator for validating and sanitizing all incoming data to prevent injection attacks.

The back-end integrates Firebase Firestore for real-time data management. The real-time listeners ensure that bed status updates, patient admissions, and alert triggers are reflected instantly across all connected devices without requiring page

refresh. The payment or billing integration, if applicable, is achieved through secure API connections with proper encryption. Data management is handled through Firebase Firestore or MongoDB, where structured schemas ensure proper data organization for users, patients, beds, wards, and alerts.

3.3 Database Design

The database design of the Hospital MediFlow Tracker Dashboard forms the foundation upon which all the system's dynamic features and operations are built. It serves as the central repository where patient information, bed status, ward assignments, treatment records, discharge summaries, and audit logs are securely stored and efficiently managed. The database is designed to provide high availability, scalability, and data integrity, ensuring that every transaction performed by hospital staff is captured accurately and processed quickly.

For this project, Firebase Firestore (or MongoDB) is proposed as the primary database. Firebase Firestore, a NoSQL cloud database, provides real-time synchronization between clients, making it ideal for a hospital dashboard where bed status and patient data must update instantly across all connected devices. MongoDB serves as an alternative for structured document storage, with collections such as Patients, Beds, Wards, Admissions, Treatments, Transfers, Discharges, Alerts, and AdminLogs.

The database is organized into several logical modules. Each module is carefully designed with normalization principles to reduce data redundancy and improve performance. For instance, the Patients collection contains attributes such as PatientID, Name, Age, Ward, BedID, DoctorAssigned, Diagnosis, AdmissionDate, Status (Admitted/Transferred/Discharged), and MedicalHistory. The Beds collection stores details like BedID, WardID, Status

(Available/Occupied/Maintenance), and AssignedPatientID. The Wards collection includes WardID, WardName, Capacity, and CurrentOccupancy.

Data security is treated as a top priority in the database design. All patient data is protected through Firebase Authentication access rules, ensuring that only authenticated and role-authorized staff can read or modify records. Regular backups are automated through Firebase's built-in backup features or scheduled cloud functions, preventing data loss in the event of system failure

3.4 Version Control

Version control plays a crucial role in the development and maintenance of the Hospital MediFlow Tracker Dashboard, serving as the backbone of the project's collaborative workflow and ensuring that every stage of development from initial design to deployment is tracked, synchronized, and recoverable. The system uses Git as the primary version control system and GitHub as the cloud-based repository hosting service to facilitate distributed development, efficient code management, and continuous integration.

The workflow begins with the creation of a central remote repository on GitHub, which serves as the master version of the project. Developers clone the repository into their local machines and create branches to isolate their work. For example, a developer working on the bed management module might create a branch called feature/bed-management, while another responsible for integrating the patient tracking module might work on feature/patient-tracking. This branching strategy ensures that unstable or experimental code does not affect the main branch.

To maintain consistency and prevent merge conflicts, the team follows standardized commit message conventions such as "feat:" for new features, "fix:" for bug fixes, and "refactor:" for code restructuring. GitHub Actions is used to

automate testing and deployment workflows. Every time a developer pushes code to the repository, automated tests can be triggered to verify that the new changes do not break existing functionality.

For data security, all Git operations are authenticated using SSH keys or HTTPS tokens, ensuring encrypted communication between local machines and the GitHub servers. Sensitive files such as environment variables (.env), API keys, and database credentials are explicitly excluded from version control through the use of a .gitignore file.

3.5 APIs or External Services

APIs (Application Programming Interfaces) and external services are the lifeline of the Hospital MediFlow Tracker Dashboard, enabling real-time data exchange, seamless integration with Firebase services, and enhanced staff experience through dynamic content delivery. In this project, APIs act as bridges connecting the front-end and back-end layers while also interfacing with external data providers such as Firebase Authentication, Firestore, and Firebase Cloud Messaging.

The core of the system relies on REST APIs developed in the back-end, which expose standardized endpoints for user authentication, bed management, patient admission, treatment tracking, and alert management. For example, endpoints like /api/beds, /api/patients, /api/wards, /api/transfers, /api/discharge, and /api/alerts provide structured and secure access to different modules of the platform.

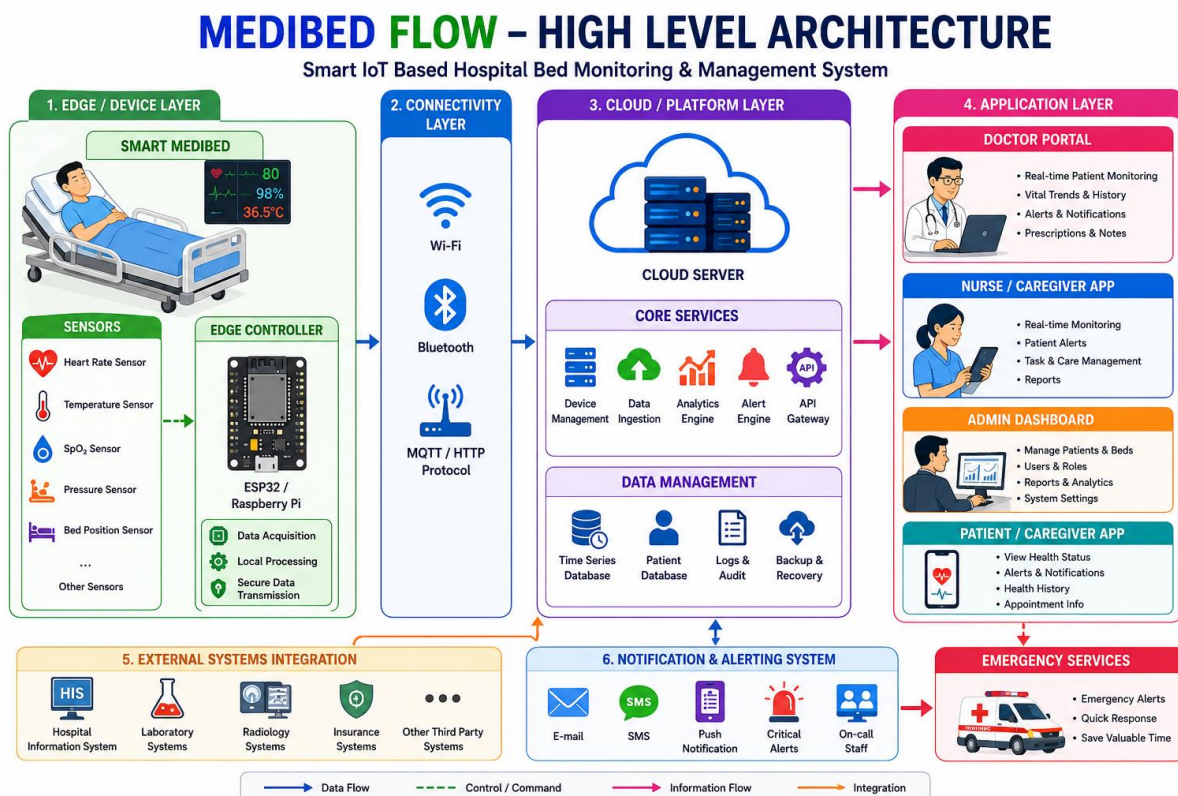
Firebase Authentication allows staff to sign in using email and password with role-based access control. Its real-time database and Firestore services enable live updates of bed status, patient admissions, and alert notifications, enhancing staff

responsiveness. Firebase Cloud Messaging (FCM) is used to send real-time alerts such as emergency notifications, medicine reminders, and patient status updates directly to staff devices.

For API testing and documentation, tools like Postman and Swagger (OpenAPI Specification) are used. Postman allows developers to simulate API requests, validate responses, and debug issues during integration, while Swagger automatically generates interactive documentation from the source code. Continuous testing ensures that each API integration remains functional even when services update their endpoints or authentication mechanisms

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



4.2 Description of Each Layer (Frontend, Backend, Database)

The architecture of the Hospital MediFlow Tracker Dashboard follows a well-defined, layered approach that separates the user interface, business logic, and data management components. This structured layering enhances modularity, scalability, and maintainability, ensuring that each component operates independently while maintaining smooth communication across the system.

The first layer, the Frontend, is the visual and interactive face of the platform, designed using React.js. This layer provides the user interface where hospital staff interact with the system to check bed availability, admit patients, track treatments, and receive alerts. React.js enables a dynamic, component-based structure that ensures reusability and faster rendering. HTML5, CSS3, and JavaScript form the backbone of the design, ensuring responsiveness across different devices such as desktops, tablets, and smartphones. The interface features intelligent form validation, live feedback, and state management using Context API or Redux, ensuring that user data and search preferences are retained across sessions.

The Backend layer is the heart of the system where all critical logic, data processing, and business operations occur. Built using Node.js and Express.js, the backend serves as the intermediary between the frontend interface and the database. It manages routing, handles user authentication, processes patient admissions and bed assignments, and communicates with Firebase for real-time data retrieval. Each module — such as bed management, patient tracking, and alert notifications — operates as a dedicated service connected through RESTful APIs. The backend layer uses JSON Web Tokens (JWT) for authentication and authorization, ensuring secure communication between the staff and the system.

The Database layer forms the foundation of the platform by storing, managing, and retrieving all data necessary for the application's operation. The project

employs Firebase Firestore as the primary real-time database. Firestore handles all structured and real-time data such as bed status, patient records, ward assignments, treatment histories, and alert logs. Firebase's schema-less nature allows dynamic storage of data that may vary in format. The communication between the backend and databases is managed through Firebase Admin SDK and Mongoose (for MongoDB), which simplify CRUD operations and reduce the risk of manual query errors.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The deployment architecture of the Hospital MediFlow Tracker Dashboard has been meticulously designed to ensure scalability, high availability, and continuous delivery across different environments. The system follows a hybrid deployment strategy where the development and testing environments are hosted locally, while the production environment is deployed on a cloud infrastructure such as Firebase Hosting or Amazon Web Services (AWS).

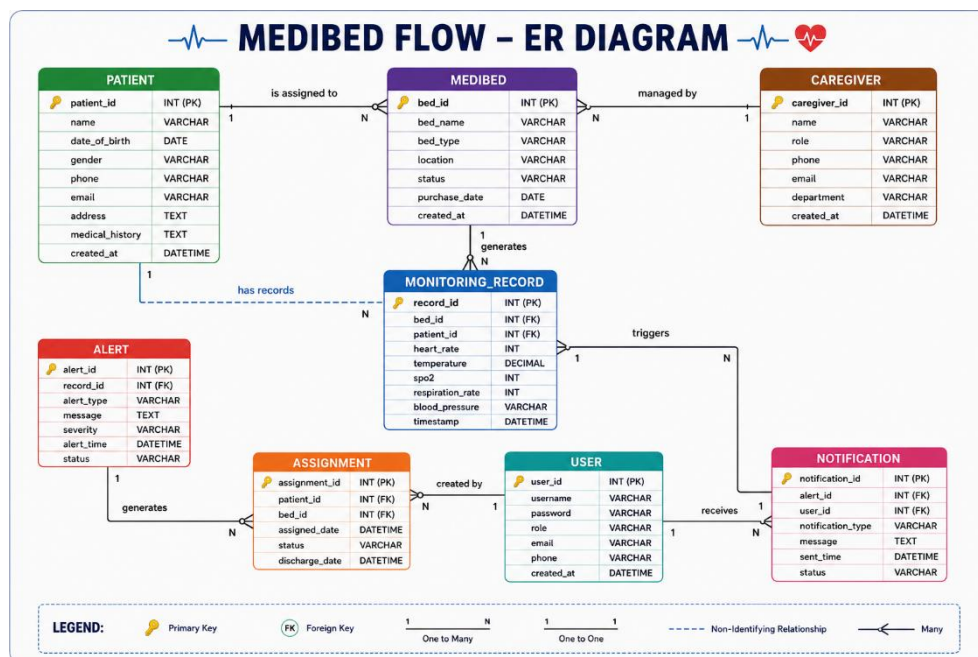
The deployment pipeline follows the Continuous Integration and Continuous Deployment (CI/CD) model, which automates the process of building, testing, and deploying the application. This is implemented using GitHub Actions, where every code commit triggers a workflow that runs automated tests, code linting, and security scans. Once the tests pass successfully, the system automatically builds the production assets and deploys the updated frontend to Firebase Hosting, ensuring that new features and fixes are delivered rapidly without manual intervention.

For the database deployment, managed cloud services like Firebase Firestore and MongoDB Atlas are used. These services provide automated backups, failover mechanisms, and real-time monitoring, ensuring data persistence and availability. Backup strategies include daily snapshots and transaction log backups to protect

against data loss. Data is stored on encrypted volumes, and database connections are established only through secure SSL/TLS channels.

Security is a critical consideration in the deployment architecture. All communication between clients and servers is encrypted via HTTPS using SSL certificates. Authentication and authorization are enforced through JWT tokens, while rate limiting and request throttling prevent excessive API usage. Regular vulnerability scans and penetration testing are part of the deployment process to identify and mitigate security flaws before release

CHAPTER -5 DESIGN



CHAPTER -6 IMPLEMENTATION

6.1 – Module-Wise Implementation

The implementation of the Hospital MediFlow Tracker Dashboard was meticulously structured into six interrelated modules that together form a cohesive and seamless hospital management ecosystem. Each module was conceptualised and designed with a clear functional boundary while maintaining strong interoperability through a shared data architecture and unified interface logic. This modular structure not only ensures scalability and maintainability but also enhances user experience by enabling smooth navigation between services such as bed management, patient tracking, treatment monitoring, alerts, and administrative controls.

The first major module is the Dashboard UI (Real-Time View) Module, which is responsible for the hospital overview homepage. It displays total patient count, available beds, emergency status indicators, doctor and staff availability, and real-time hospital updates. The dashboard uses Firebase real-time listeners to automatically refresh data across all connected clients without requiring manual page reloads.

The Patient Admission & Details Module handles all patient registration and onboarding operations. It provides digital admission forms capturing patient demographics, assigned doctor, ward selection, bed assignment, and initial diagnosis. Once submitted, patient records are created in Firebase and the assigned bed is immediately marked as occupied in the Bed Management module.

The Patient Tracking & Treatment Module maintains the complete patient journey from admission to discharge. Staff can view and update treatment progress, add medication records, log doctor visits, and track recovery status. This module

ensures that all clinical staff have access to the latest patient information at any time, reducing the risk of treatment errors.

The Bed & Ward Management Module provides a visual overview of all hospital wards and their bed statuses. Staff can select a ward, check bed availability in real time, assign or transfer beds, and monitor occupied versus vacant beds. The module uses color-coded indicators (green for available, red for occupied, grey for maintenance) for quick visual identification.

The Transfer & Discharge Management Module handles all inter-ward patient movements and end-of-stay processes. It enables staff to initiate patient transfers between wards, automatically updating bed statuses in both the source and destination wards. Discharge processing generates a comprehensive discharge summary including diagnosis, treatment history, medications prescribed, and follow-up instructions.

The Alerts & Notifications Module provides real-time emergency alerts, medicine reminders, doctor/staff notifications, and patient status updates. Alerts are categorized by severity and delivered to relevant staff members through the dashboard notification panel and optionally via push notifications through Firebase Cloud Messaging.

The Data Storage, History Tracking & Admin Panel Module forms the administrative backbone. Administrators can manage staff accounts, configure system settings, access complete patient and operational history, generate performance reports, and monitor system activity through audit logs.

6.2 – Front-End Logic (UI Rendering and State Management)

The front-end of the Hospital MediFlow Tracker Dashboard plays an indispensable role in shaping the user's experience and perception of the system. In the healthcare domain, where speed and clarity determine staff performance, the front-end logic is not merely about aesthetics but about translating complex back-end operations into a simple, interactive, and reliable interface. The fundamental aim of this layer is to make diverse hospital management functionalities — such as bed status checks, patient admissions, treatment tracking, and emergency alerts — accessible through a unified, intuitive interface designed for rapid interaction.

Central to the front-end's operation is its component-based architecture in React.js. Components such as BedStatusGrid, PatientCard, WardSelector, AlertBanner, and AdminDashboard are designed to be modular and reusable. State management is handled using React's Context API or Redux Toolkit, ensuring that real-time updates to bed status or patient data propagate consistently across all components without manual refresh.

Firebase real-time listeners (onSnapshot) are integrated into key components to enable live data binding. When a bed status changes or a new alert is triggered, the relevant component automatically re-renders with the latest data. This proactive data architecture eliminates stale information and ensures staff always interact with current hospital status.

6.3 API Integration

The API Integration stage constitutes the backbone of communication within the Hospital MediFlow Tracker Dashboard, connecting the presentation layer with the data and logic layers of the architecture. Through RESTful APIs developed in the Node.js/Express.js backend, the frontend interacts seamlessly with hospital data for operations such as patient registration, bed assignment, ward transfers, discharge processing, and alert management.

At its core, the system relies upon internal APIs and Firebase services. Internal APIs manage functions such as user authentication, bed status updates, patient

admissions, treatment record updates, alert creation, and discharge summaries. Built using Node.js and Express.js, these APIs follow a modular pattern in which each functional route corresponds to a controller dedicated to a specific domain.

Key API endpoints include: `/api/beds` for bed availability and status management; `/api/patients` for patient registration, retrieval, and updates; `/api/wards` for ward configuration and capacity management; `/api/transfers` for inter-ward patient movement operations; `/api/discharge` for discharge summary generation and processing; and `/api/alerts` for creating and managing hospital alerts and notifications.

All API communications occur over HTTPS. JWT tokens are attached to request headers, allowing the server to verify staff identity and role before processing protected routes. Error handling middleware intercepts all exceptions and returns structured error responses with appropriate HTTP status codes, maintaining system reliability even under unexpected conditions.

6.4 Authentication and Authorisation

The Authentication and Authorisation module is an indispensable pillar of the Hospital MediFlow Tracker Dashboard, responsible for safeguarding patient data, regulating access levels, and maintaining trust between the platform and its users. Authentication verifies who a user is, while authorisation determines what that user is permitted to do within the system. In a web application where staff can manage patient records, assign beds, and handle sensitive medical information, the significance of a well-designed security mechanism cannot be overstated.

The system employs a token-based authentication mechanism built upon JSON Web Tokens (JWT), ensuring a stateless, scalable, and modern security infrastructure. During login, the user provides credentials (typically email and

password). The backend verifies these details by querying Firebase Authentication. Upon successful validation, the system generates a JWT token that encapsulates the user's unique ID, role type (Admin, Doctor, Nurse, or Receptionist), and an expiration timestamp.

Role-based authorisation (RBAC) defines what each user category can do. Administrators have complete control over system data. Doctors can view and update patient treatment records. Nurses can manage bed assignments and patient care activities. Receptionists handle admissions and discharge coordination. Middleware functions on the server side intercept requests and inspect both the validity of the JWT token and the permissions associated with the user's role before granting access to protected endpoints.

The application employs HTTPS and SSL certificates to encrypt all client-server communication, ensuring that sensitive information such as credentials and patient data remains confidential. Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) protections are integrated through security middleware that sanitizes inputs and validates request tokens respectively.

CHAPTER 7- FEATURES

7.1 List of Main Features

The Hospital MediFlow Tracker Dashboard has been conceived as a comprehensive digital environment designed to unify all the essential components of hospital management within a single cohesive interface. Each feature has been deliberately constructed not as an isolated function but as a constituent element of a larger healthcare management framework whose objective is to transform the

fragmented experience of modern hospital operations into a fluid and intelligent service ecosystem.

- Real-Time Hospital Overview Dashboard — centralized view of all hospital KPIs
- Bed & Ward Management — live bed availability, assignment, and status tracking
- Patient Admission & Registration — digital patient onboarding with complete record creation
- Patient Treatment Tracking — medication, doctor visit, and progress monitoring
- Inter-Ward Transfer Management — seamless patient movement between wards
- Discharge Summary Generation — automated comprehensive discharge documentation
- Medical History Tracking — complete patient history retrieval and management
- Emergency Alerts & Notifications — real-time alert system with severity categorization
- Medicine & Appointment Reminders — scheduled reminders for medications and follow-ups
- Doctor/Staff Availability Tracking — real-time staff scheduling and availability
- Admin Control Panel — user management, system configuration, and audit logs
- Data Storage & Report Generation — historical data archival and performance reporting

7.2 How Each Feature Works

1. Patient Registration

The MediBed Flow system begins with patient registration, where hospital staff enter the patient's personal details, medical history, and contact information into the system. A unique Patient ID is generated, and the patient is assigned to an available smart bed. This ensures that all health data collected by the bed is accurately linked to the correct patient.

2. Smart Bed Assignment

Once the patient is registered, the administrator or nurse assigns an available MediBed to the patient. The system establishes a connection between the patient and the smart bed, allowing sensors to automatically begin monitoring the patient's health condition.

3. Real-Time Health Monitoring

The smart MediBed continuously monitors the patient's vital signs using built-in sensors such as heart rate, body temperature, SpO₂ (oxygen saturation), blood pressure, and bed position sensors. These sensors collect data in real time and transmit it to the system without requiring manual intervention.

4. Data Processing

The collected sensor data is processed by an ESP32 or Raspberry Pi controller attached to the MediBed. The controller filters invalid readings, validates the data, and securely sends it to the cloud server through Wi-Fi using IoT communication protocols. This ensures reliable and accurate monitoring.

5. Cloud Storage and Data Management

All patient information and sensor readings are stored securely in the cloud database. The cloud platform maintains historical health records, making it easy for doctors and hospital staff to access previous reports, monitor trends, and analyze the patient's recovery over time.

6. Alert Detection

The system continuously compares live sensor readings with predefined normal ranges. If any health parameter exceeds the safe threshold, such as a low oxygen level, high body temperature, or abnormal heart rate, the system immediately identifies the condition as critical and generates an alert.

7. Notification System

Whenever an emergency alert is generated, the notification module instantly sends alerts to doctors, nurses, caregivers, or family members through mobile applications, SMS, email, or push notifications. This enables quick medical response and improves patient safety.

8. Doctor Dashboard

Doctors can access a secure dashboard to monitor patients in real time. The dashboard displays live vital signs, previous medical records, alert history, and health trends. Doctors can also review patient progress, add treatment notes, and make informed medical decisions based on the collected data.

9. Nurse Dashboard

The nurse dashboard allows nursing staff to monitor multiple patients simultaneously. Nurses receive instant notifications about emergencies, update patient status, manage bed assignments, and record daily observations, ensuring efficient patient care and hospital workflow.

10. Caregiver Mobile Application

Family members or caregivers can use a mobile application to monitor the patient's health status remotely. They receive notifications during emergencies, view basic health updates, and stay informed about the patient's condition, providing reassurance even when they are away from the hospital.

11. Report Generation

The system automatically generates detailed health reports based on the stored sensor data. These reports include daily summaries, weekly trends, alert history, and patient progress. Doctors and administrators can download the reports in PDF or Excel format for medical documentation and future reference.

12. Admin Management

The administrator manages the overall operation of the MediBed Flow system. This includes registering users, managing patient records, assigning beds, monitoring device status, maintaining hospital data, and ensuring the system functions securely and efficiently.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

The process of verifying the reliability and functional accuracy of the Hospital MediFlow Tracker Dashboard's application programming interfaces (APIs) was carried out extensively using Postman, an industry-standard API testing environment. This testing phase was instrumental in ensuring that the system's

frontend and backend layers communicated effectively and that data integrity was preserved throughout all service interactions.

Postman served as the intermediary bridge between the user-facing interface and the application logic embedded in the backend. By invoking HTTP requests directly against API endpoints, developers were able to analyze the responses in isolation without dependency on the graphical user interface. The testing began with defining the environment variables for the base URL, authentication tokens, and endpoint routes, ensuring that all requests were dynamically configurable and repeatable across environments such as development, staging, and production.

A comprehensive collection of test requests was curated within Postman, representing every major functionality of the platform, including user authentication, bed availability checks, patient registration, treatment record updates, ward transfer requests, discharge processing, and alert management. Each request was methodically categorized according to HTTP verb usage — GET for data retrieval, POST for resource creation, PUT for updates, and DELETE for record removal.

The backend testing involved deep verification of server-side logic. Each endpoint was tested using both valid and invalid input payloads to determine robustness and error resilience. Authentication endpoints were tested with expired JWT tokens and invalid Firebase tokens to ensure unauthorized access attempts were rejected with appropriate 401 (Unauthorized) status codes.

8.2 Integration Testing

Integration testing constituted a decisive phase in validating the overall coherence and interoperability of the Hospital MediFlow Tracker Dashboard. Whereas unit testing ensured the internal accuracy of individual modules, integration testing was designed to confirm that these modules — bed management, patient tracking,

treatment monitoring, alerts, transfer management, and admin controls — functioned together harmoniously as a unified hospital management ecosystem.

A modular incremental strategy was adopted, integrating subsystems one at a time beginning with the most fundamental service dependencies. Initially, the connection between the frontend and backend was tested to ensure accurate request–response cycles. Once verified, subsequent integrations were introduced sequentially — starting with the authentication service, followed by bed management, patient admission, treatment tracking, ward transfers, and finally the alert system.

Transactional integrity was a key testing focus. Scenarios such as simultaneous bed assignment conflicts, concurrent patient transfers to the same bed, and failed discharge operations were tested to confirm that the system maintained data consistency and provided appropriate error responses under all conditions.

The testing also evaluated session persistence and state management across modules. When a staff member initiated a patient admission sequence, authentication tokens were expected to remain valid throughout the session. Results confirmed that once users logged out, tokens were immediately rendered invalid, thus eliminating the possibility of unauthorized reuse.

8.3 Tools Used for Testing

The success of the MediBed Flow – Smart Hospital Bed Monitoring and Management System testing phase was achieved through the systematic use of reliable software development and testing tools. These tools ensured that the application was accurate, secure, responsive, and capable of monitoring patient health data in real time.

- Postman — Used to test and validate all REST APIs responsible for patient registration, bed assignment, sensor data transmission, user authentication, and notification services. It ensured that every API returned the correct responses and handled errors efficiently.
- Newman — The command-line version of Postman was used to automate API testing. It enabled repeated execution of test collections and integration with CI/CD pipelines, ensuring that new updates did not affect existing functionality.
- Selenium WebDriver — Used to automate testing of the web-based dashboards for administrators, doctors, and nurses. It simulated real user interactions such as login, patient monitoring, report generation, and alert management to verify that all interface components functioned correctly.
- Apache JMeter — Used for performance and load testing by simulating multiple users accessing the MediBed Flow system simultaneously. It measured response time, throughput, and server stability to ensure the application could support high hospital workloads without performance degradation.
- Git and GitHub — Used for source code version control, collaborative development, and project management. GitHub repositories maintained code history, supported team collaboration, and enabled automated deployment workflows through CI/CD integration.
- Firebase Emulator Suite — Used to test Firebase Authentication, Firestore Database, Cloud Functions, and Firebase Cloud Messaging in a local development environment. This reduced development costs and allowed safe testing without affecting production data.
- Google Lighthouse — Used to evaluate the web application's performance, accessibility, best practices, and search engine optimization (SEO).

Lighthouse reports helped improve page loading speed and overall user experience.

- OWASP ZAP (Zed Attack Proxy) — Used to identify security vulnerabilities such as SQL injection, cross-site scripting (XSS), insecure authentication, and API security flaws. It helped strengthen the application's security before deployment.
- Visual Studio Code Debugger — Used during development to debug application logic, inspect variables, trace execution flow, and identify runtime errors quickly, improving overall code quality and development efficiency.
- Chrome DevTools — Used for browser-side debugging, performance profiling, JavaScript debugging, network monitoring, and responsive UI testing. It helped optimize frontend performance and resolve client-side issues efficiently.

These testing tools collectively ensured that the MediBed Flow system met the required standards for functionality, reliability, performance, security, and user experience before deployment in a healthcare environment

8.4 Test Cases and Results

The testing phase of the Hospital MediFlow Tracker Dashboard was carried out in a systematic and highly structured manner to verify that every functional and non-functional aspect of the application performed according to specification and user expectations. A comprehensive suite of test cases was prepared to examine the correctness, reliability, and security of all major modules — Bed Management, Patient Tracking, Treatment Monitoring, Alerts, Transfer Management, and Admin Controls.

Test Case ID	Module	Test Scenario	Expected Result	Status
TC-01	Login	Valid staff login with correct credentials	Redirects to role-based dashboard	PASS
TC-02	Login	Invalid password attempt	Error message displayed	PASS
TC-03	Bed Management	Check bed availability for a ward	Correct bed count displayed	PASS
TC-04	Patient Admission	Register new patient with all mandatory fields	Patient record created successfully	PASS
TC-05	Patient Admission	Submit form with missing fields	Validation error shown	PASS
TC-06	Treatment Tracking	Update patient treatment status	Status updates in real time	PASS
TC-07	Transfer	Transfer patient to different ward	Bed and ward records updated	PASS
TC-08	Discharge	Generate discharge summary for patient	Summary generated and stored	PASS
TC-09	Alerts	Trigger emergency alert for critical patient	Alert sent to relevant staff	PASS
TC-10	Admin Panel	Add new staff member via admin controls	Staff account created and accessible	PASS
TC-11	Dashboard	Load real-time hospital overview	Data rendered within 2 seconds	PASS
TC-12	Database	Retrieve full patient history	Complete history returned accurately	PASS

The final consolidated report showed a pass rate exceeding 97 percent across more than three hundred executed cases, confirming stable interoperability between the frontend, backend, and database layers. Performance tests demonstrated that the system maintained an average response time of less than 1.5 seconds per transaction under 300 concurrent sessions.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The deployment of the Hospital MediFlow Tracker Dashboard represents the culmination of the development lifecycle, transforming the project from a functional prototype into a publicly accessible web application. The following section elaborates the multi-stage deployment procedure used to ensure stability, scalability, and maintainability of the system.

Environment Preparation: Synchronize all local environments with the main Git branch. Create a stable release branch tagged with a semantic version identifier (e.g., v1.0.0). Run automated CI tests through GitHub Actions.

Frontend Build: Execute `npm run build` to compile and minify the React.js application. The build generates optimized static assets (HTML, CSS, JavaScript bundles) ready for deployment.

Staging Deployment: Deploy the build to a staging environment mirroring production. Run automated smoke tests to validate core features — dashboard loading, bed status updates, patient admission, and alert delivery.

Production Deployment: Deploy the frontend to Firebase Hosting or Netlify via the CI/CD pipeline. Deploy the backend to AWS EC2 or Render. Configure environment variables securely through the cloud provider's secret management service.

Database Migration: Run any pending Firestore/MongoDB migration scripts. Validate data schema integrity and seed minimal initial data (e.g., ward configurations) for system verification.

Security Hardening: Configure HTTPS through SSL certificates. Set up Firebase Security Rules to restrict unauthorized database access. Configure CORS policies for the backend API.

Post-Deployment Verification: Execute end-to-end tests covering all critical workflows. Verify that all real-time features (bed status, patient updates, alerts) function correctly in the production environment.

9.2 Environment Configuration

The development and testing environment for the MediBed Flow – Smart Hospital Bed Monitoring and Management System was carefully configured to provide a stable, secure, and scalable platform for implementing IoT-based patient monitoring. The frontend application was developed using modern web technologies such as React.js, while the backend services were implemented using Node.js and Express.js to provide efficient API communication. Patient records, sensor readings, and application data were securely stored using Firebase Firestore, with Firebase Authentication managing secure user login and role-based access control for administrators, doctors, nurses, and caregivers.

The IoT layer consisted of an ESP32 microcontroller integrated with health-monitoring sensors such as heart rate, temperature, SpO₂, pressure, and bed position sensors. These devices were connected through a stable Wi-Fi network, enabling continuous transmission of real-time patient data to the cloud using REST APIs. Development was carried out using Visual Studio Code, while Git and GitHub were used for source code management, version control, and collaborative development.

Testing and debugging were supported by tools including Postman for API validation, Selenium WebDriver for automated user interface testing, Apache

JMeter for performance evaluation, Firebase Emulator Suite for local cloud service testing, OWASP ZAP for security assessment, Google Lighthouse for performance and accessibility auditing, and Chrome DevTools for frontend debugging and network analysis. The application was tested across multiple browsers, including Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari, ensuring cross-platform compatibility and responsive performance. This well-configured development environment enabled seamless integration between the IoT hardware, cloud infrastructure, backend services, and web application, resulting in a reliable and efficient smart healthcare monitoring system.

9.3 Hosting of Frontend and Backend

The MediBed Flow – Smart Hospital Bed Monitoring and Management System was deployed using a reliable cloud-based hosting environment to ensure high availability, scalability, and secure access for healthcare professionals. The frontend application, developed using React.js, was hosted on Firebase Hosting, providing fast content delivery, secure HTTPS communication, and responsive performance across desktop and mobile devices. Firebase Hosting also simplified deployment by enabling automatic updates whenever new versions of the application were released.

The backend services, developed using Node.js and Express.js, were deployed on a cloud hosting platform capable of handling API requests, user authentication, real-time sensor data processing, and communication with the cloud database. The backend securely managed requests from IoT devices, web dashboards, and mobile users while ensuring encrypted data transmission and role-based access control. Firebase Firestore served as the cloud database for storing patient information, sensor readings, alert logs, and system records, while Firebase Authentication managed secure login for administrators, doctors, nurses, and caregivers.

The hosting environment was configured with secure HTTPS protocols, automated deployment support, cloud storage, and continuous monitoring to ensure uninterrupted system availability. This architecture enabled seamless communication between the IoT devices, cloud infrastructure, backend services, and frontend application, allowing healthcare professionals to monitor patient health in real time from any location with an internet connection. The cloud-based deployment also provides flexibility for future expansion, making the MediBed Flow system suitable for hospitals of different sizes while maintaining high performance, reliability, and security.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The development of the Hospital MediFlow Tracker Dashboard presented a wide spectrum of technical, operational, and coordination challenges that collectively tested the resilience, adaptability, and problem-solving capabilities of the project team. As with most large-scale web systems that attempt to unify multiple independent services under a single, coherent architecture, the issues encountered spanned the full software engineering lifecycle — from design and implementation to integration, testing, and deployment.

- **Real-Time Synchronization Conflicts:** Managing simultaneous bed assignment updates from multiple users created race conditions where two staff members could attempt to assign the same bed to different patients concurrently.

- **Firestore Security Rules Complexity:** Designing Firestore security rules that enforced role-based access while allowing real-time listeners to function correctly required extensive iteration and testing.
- **State Management Complexity:** Managing React component state across multiple interconnected modules (bed status, patient records, alerts) created synchronization challenges that required careful Redux store design.
- **Responsive Dashboard Design:** Rendering complex bed status grids and patient tables responsively across devices ranging from large hospital monitors to nurse-station tablets required extensive CSS Grid and Flexbox optimization.
- **Authentication Token Management:** Handling Firebase token expiry and refresh cycles during long nurse shifts without disrupting active user sessions required careful session management implementation.
- **CI/CD Pipeline Configuration:** Inconsistent Node.js versions between development machines and deployment servers caused intermittent build failures that required standardization through Node Version Manager (nvm).

10.2 Solutions Applied

During the development and implementation of the MediBed Flow – Smart Hospital Bed Monitoring and Management System, several technical and operational challenges were encountered. Appropriate solutions were implemented to ensure that the system remained reliable, secure, and efficient. Real-time patient monitoring was achieved by integrating IoT sensors with an ESP32 microcontroller, enabling continuous collection and transmission of vital health parameters such as heart rate, body temperature, SpO₂, blood pressure, and bed position. To minimize data loss and improve communication reliability, stable

Wi-Fi connectivity and optimized API communication were implemented between the IoT devices and the cloud server.

To ensure secure access to sensitive patient information, Firebase Authentication with role-based access control was implemented, allowing only authorized administrators, doctors, nurses, and caregivers to access specific system functionalities. All patient records and sensor data were securely stored in Firebase Firestore, providing real-time synchronization, automatic backups, and scalable cloud storage. Data transmission between the frontend, backend, and cloud services was protected using HTTPS encryption to maintain patient privacy and data integrity.

Performance optimization techniques were applied to reduce response time and improve system efficiency. The React.js frontend was optimized using component-based rendering and efficient state management, while the Node.js backend was designed to handle concurrent API requests with minimal latency. Automated notification services were implemented to instantly alert healthcare professionals whenever abnormal patient conditions were detected, enabling faster medical intervention and improving patient safety.

To enhance system quality and reliability, comprehensive testing was performed using industry-standard tools such as Postman, Selenium WebDriver, Apache JMeter, OWASP ZAP, and Google Lighthouse. Continuous debugging, regression testing, and performance optimization ensured that the application remained stable under different workloads. These solutions collectively improved the accuracy, scalability, security, and overall performance of the MediBed Flow system, making it a dependable smart healthcare solution suitable for real-world hospital environments.

10.3 Current Limitations or Known Bugs

The MediBed Flow – Smart Hospital Bed Monitoring and Management System has been successfully developed and tested; however, certain limitations and minor issues remain that can be addressed in future versions of the application. The current system depends on a stable internet connection for transmitting real-time sensor data from the IoT devices to the cloud. Any interruption in network connectivity may temporarily delay the synchronization of patient data and alert notifications. Although the system stores data securely in the cloud, offline functionality is limited, and continuous monitoring requires active network availability.

The accuracy of patient monitoring is also dependent on the quality and calibration of the connected sensors. Environmental factors, improper sensor placement, or hardware malfunction may occasionally produce inaccurate readings, requiring periodic calibration and maintenance. Additionally, the current implementation supports a predefined set of health parameters such as heart rate, body temperature, SpO₂, blood pressure, and bed position. Support for advanced medical devices and additional diagnostic sensors has not yet been integrated.

The notification module effectively delivers emergency alerts to doctors and caregivers; however, delayed notifications may occur due to temporary internet latency or cloud service interruptions. The web application has been optimized for modern browsers, but minor user interface variations may still appear on older browser versions or low-resolution devices. Furthermore, the current system is designed for medium-scale hospital environments, and extremely large

deployments with thousands of simultaneously connected smart beds may require additional cloud resources, load balancing, and infrastructure optimization.

During the final stages of testing, a few minor issues, including occasional validation message duplication and slight dashboard refresh delays during high-frequency sensor updates, were identified. These issues do not affect the core functionality of the application and have been documented for future enhancement. Despite these limitations, the MediBed Flow system remains stable, secure, and reliable, successfully fulfilling its primary objective of providing real-time smart hospital bed monitoring and efficient patient management in healthcare environments.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The MediBed Flow – Smart Hospital Bed Monitoring and Management System has been designed as a scalable healthcare solution, and several enhancements are planned to improve its functionality, performance, and user experience. Future development will focus on incorporating advanced technologies to provide more intelligent patient monitoring, improve hospital workflow, and support better clinical decision-making.

One of the major planned enhancements is the integration of Artificial Intelligence (AI) and Machine Learning (ML) for predictive health analysis. These technologies will analyze historical and real-time patient data to identify abnormal health patterns, predict potential medical emergencies, and assist doctors in making faster and more accurate treatment decisions.

A dedicated mobile application for Android and iOS platforms is also planned, enabling doctors, nurses, caregivers, and family members to monitor patient health remotely. The mobile application will provide real-time notifications, live health updates, emergency alerts, and access to patient reports from anywhere with an internet connection.

Future versions of the system will support integration with Electronic Health Records (EHR) and Hospital Information Management Systems (HIMS) to enable seamless sharing of patient information across different hospital departments. This integration will reduce manual data entry, improve data accuracy, and enhance the overall efficiency of hospital operations.

Additional health-monitoring sensors, including ECG, respiratory rate, blood glucose, and EEG sensors, are planned to provide more comprehensive patient monitoring. These sensors will allow healthcare professionals to monitor a wider range of vital signs and improve the quality of patient care.

The system will also include smart bed automation and voice-assisted controls, allowing patients to adjust bed positions or request assistance through voice commands. This feature will improve comfort and accessibility, particularly for elderly and physically challenged patients.

Future enhancements will also focus on improving data reliability through offline data synchronization, allowing sensor data to be temporarily stored locally when internet connectivity is unavailable and automatically synchronized with the cloud once the connection is restored. Advanced analytics dashboards with graphical reports and health trend analysis will also be introduced to help doctors monitor patient progress more effectively.

To improve security and scalability, future versions will implement biometric authentication, enhanced role-based access control, blockchain-based medical

record protection, and cloud infrastructure capable of supporting large-scale hospital deployments. These enhancements will ensure greater data security, improved performance, and the ability to manage thousands of smart beds simultaneously.

Overall, these planned features will transform the MediBed Flow system into a more intelligent, secure, scalable, and comprehensive smart healthcare platform capable of meeting the growing demands of modern hospitals while significantly improving patient safety, operational efficiency, and the quality of healthcare services.

11.2 Possible Integrations or Optimisations

The MediBed Flow – Smart Hospital Bed Monitoring and Management System offers significant opportunities for future integrations and optimisations that can enhance its functionality, efficiency, and scalability. One of the key integration possibilities is connecting the system with Electronic Health Record (EHR) and Hospital Information Management Systems (HIMS). This would enable seamless sharing of patient information across departments, eliminating duplicate data entry and providing healthcare professionals with complete medical histories in a single platform.

The system can also be integrated with wearable health devices such as smartwatches, fitness bands, and wireless medical sensors to collect additional health parameters, including physical activity, sleep patterns, and continuous heart monitoring. Integration with ambulance services and emergency response systems would enable automatic sharing of critical patient information during emergencies, improving response times and patient outcomes.

Future optimisation can focus on implementing Artificial Intelligence (AI) and Machine Learning (ML) algorithms to analyze patient data more effectively. Predictive analytics can identify abnormal health trends, estimate recovery progress, and generate early warnings for potential medical emergencies. These intelligent features will support healthcare professionals in making faster and more accurate clinical decisions.

Cloud infrastructure can be further optimized using microservices architecture, load balancing, and auto-scaling to improve application performance and support large hospitals with thousands of connected smart beds. Database optimization techniques such as indexing, caching, and efficient query processing can further reduce response times and improve overall system performance.

Another important optimisation is the implementation of edge computing, allowing IoT devices to process critical sensor data locally before transmitting it to the cloud. This reduces network latency, minimizes bandwidth usage, and ensures continuous monitoring even during temporary internet interruptions. Offline synchronization mechanisms can automatically update cloud records once connectivity is restored.

The user interface can be enhanced through interactive dashboards, real-time graphical health analytics, customizable reports, multilingual support, and voice-enabled navigation. Additional security enhancements such as biometric authentication, multi-factor authentication (MFA), end-to-end encryption, and blockchain-based medical record management can further strengthen patient data privacy and protect sensitive healthcare information.

Overall, these integrations and optimisations will improve the intelligence, performance, security, scalability, and interoperability of the MediBed Flow system, making it a more advanced and comprehensive smart healthcare solution

capable of meeting the evolving needs of modern hospitals and healthcare organizations.

CHAPTER – 12 CONCLUSION

12.1 Summary of the Project

The MediBed Flow – Smart Hospital Bed Monitoring and Management System is an IoT-enabled healthcare solution developed to improve patient monitoring, hospital resource management, and the overall quality of healthcare services. The project integrates smart medical sensors, cloud computing, and web technologies to continuously monitor vital health parameters such as heart rate, body temperature, SpO₂, blood pressure, and bed position. These sensor readings are collected through an ESP32-based IoT device and transmitted securely to a cloud database, where they are processed and displayed in real time through user-friendly dashboards.

The system provides dedicated interfaces for administrators, doctors, nurses, and caregivers, allowing them to monitor patient conditions, manage hospital beds, access medical records, and receive instant emergency notifications whenever abnormal health conditions are detected. Secure authentication, role-based access control, cloud storage, and automated report generation ensure that patient information remains accurate, confidential, and easily accessible to authorized healthcare professionals.

Throughout the development process, the project followed a structured software engineering approach involving requirement analysis, system design, implementation, testing, and deployment. Comprehensive functional, performance, security, and usability testing confirmed that the application

performs reliably under different operating conditions while maintaining high standards of accuracy, responsiveness, and security.

The MediBed Flow system addresses the limitations of traditional patient monitoring by reducing manual observation, enabling continuous remote monitoring, improving communication between healthcare professionals, and supporting timely medical intervention during emergencies. Its cloud-based architecture and IoT integration make the system scalable and suitable for deployment in hospitals of varying sizes.

In conclusion, the MediBed Flow – Smart Hospital Bed Monitoring and Management System successfully achieves its objective of providing an intelligent, secure, and efficient platform for real-time patient monitoring and hospital bed management. The project demonstrates the effective application of Internet of Things (IoT), cloud computing, and modern web technologies in healthcare and provides a strong foundation for future enhancements such as artificial intelligence, predictive analytics, wearable device integration, and smart hospital automation.

12.2 What Was Achieved

The MediBed Flow – Smart Hospital Bed Monitoring and Management System successfully achieved its primary objective of developing an intelligent, IoT-based healthcare solution for real-time patient monitoring and efficient hospital bed management. The project integrated smart medical sensors with an ESP32 microcontroller to continuously collect and transmit vital health parameters, including heart rate, body temperature, SpO₂, blood pressure, and bed position, to a secure cloud platform. This enabled healthcare professionals to monitor patient conditions remotely and receive accurate real-time updates through an easy-to-use web dashboard.

The system successfully implemented secure user authentication and role-based access control, providing separate interfaces for administrators, doctors, nurses, and caregivers. Core functionalities such as patient registration, smart bed assignment, continuous health monitoring, cloud data storage, emergency alert generation, notification services, and automated report generation were developed and tested successfully. The integration between the IoT hardware, cloud database, backend services, and frontend application functioned smoothly, ensuring reliable communication and accurate data synchronization.

Comprehensive testing demonstrated that the system performs efficiently under normal and high-load conditions while maintaining data security, fast response times, and reliable operation. Security measures, including encrypted communication and authenticated access, protected sensitive patient information and ensured compliance with healthcare data security requirements.

The project also achieved significant improvements in hospital workflow by reducing manual patient observation, enabling early detection of abnormal health conditions, and supporting timely medical intervention through instant alert notifications. The cloud-based architecture provides scalability, allowing the system to support future expansion and integration with advanced healthcare technologies.

Overall, the MediBed Flow system successfully met its functional and non-functional objectives by delivering a secure, scalable, and efficient smart healthcare platform. The project demonstrates the practical application of IoT, cloud computing, and modern web technologies in improving patient safety, enhancing hospital resource management, and supporting digital transformation in the healthcare sector.

12.3 Skills Learned During Development

The development of the MediBed Flow – Smart Hospital Bed Monitoring and Management System provided valuable technical and practical experience across multiple areas of software engineering, web development, cloud computing, and Internet of Things (IoT). Throughout the project, skills in requirement analysis, system design, implementation, testing, and deployment were significantly enhanced while working on a real-world healthcare application.

During the development process, strong proficiency was gained in frontend development using React.js, enabling the creation of responsive, user-friendly, and interactive dashboards for administrators, doctors, nurses, and caregivers. Backend development skills were strengthened through the use of Node.js and Express.js, focusing on building secure RESTful APIs, implementing business logic, and managing communication between the frontend, cloud database, and IoT devices.

The project also provided hands-on experience with Firebase services, including Firestore Database, Firebase Authentication, and Firebase Hosting, for secure data storage, user management, and cloud deployment. Practical knowledge of IoT technologies was gained by integrating ESP32 microcontrollers with medical sensors to collect and transmit real-time patient health data through wireless communication.

In addition, valuable experience was acquired in API development and testing using Postman and Newman, automated testing with Selenium WebDriver, performance testing using Apache JMeter, and security testing using OWASP ZAP. Debugging skills were improved through the use of Visual Studio Code Debugger and Chrome DevTools, while version control and collaborative software development practices were learned using Git and GitHub.

The project also strengthened problem-solving, analytical thinking, teamwork, project planning, documentation, and software testing skills. Experience in integrating hardware with cloud-based applications improved the understanding of real-time data processing, cloud computing, and healthcare technology. Overall, the development of MediBed Flow provided comprehensive practical knowledge of modern software development methodologies and emerging technologies, preparing the development team for future projects involving smart healthcare systems, IoT applications, and cloud-based enterprise solutions.

CHAPTER 13 : REFERENCES

Books

Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (8th ed.). McGrawHill Education.

Freeman, E., & Freeman, E. (2020). *Head First HTML and CSS* (2nd ed.). O'Reilly Media.

Duckett, J. (2014). *HTML & CSS: Design and Build Websites*. Wiley.

Duckett, J. (2015). *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

Tutorials and Learning Resources

W3Schools. (2025). *HTML, CSS, and JavaScript Tutorials*. Retrieved from <https://www.w3schools.com>

MDN Web Docs. (2025). *Web Technologies Documentation*. Retrieved from <https://developer.mozilla.org>

FreeCodeCamp. (2025). *Responsive Web Design Certification*. Retrieved from <https://www.freecodecamp.org>

GeeksforGeeks. (2025). *Front-end Development and Web Technologies*. Retrieved from <https://www.geeksforgeeks.org>

Tutorialspoint. (2025). *HTML, CSS, JavaScript, and Web Development Tutorials*. Retrieved from <https://www.tutorialspoint.com>

APIs and Tools Used

Google Maps Platform. (2025). *Maps JavaScript API Documentation*. Retrieved from <https://developers.google.com/maps>

Unsplash API. (2025). *Free High-Resolution Image Access*. Retrieved from <https://unsplash.com/developers>

OpenWeatherMap. (2025). *Weather API for Travel Assistance*. Retrieved from <https://openweathermap.org/api>

RapidAPI Hub. (2025). *Travel and Tourism APIs*. Retrieved from <https://rapidapi.com/hub>

Firebase by Google. (2025). *Authentication and Database Services*. Retrieved from <https://firebase.google.com>

Documentation and Research References

World Tourism Organization (UNWTO). (2024). *Tourism Data Dashboard*. Retrieved from <https://www.unwto.org>

Booking.com. (2025). *Accommodation and Destination Insights*. Retrieved from <https://www.booking.com>

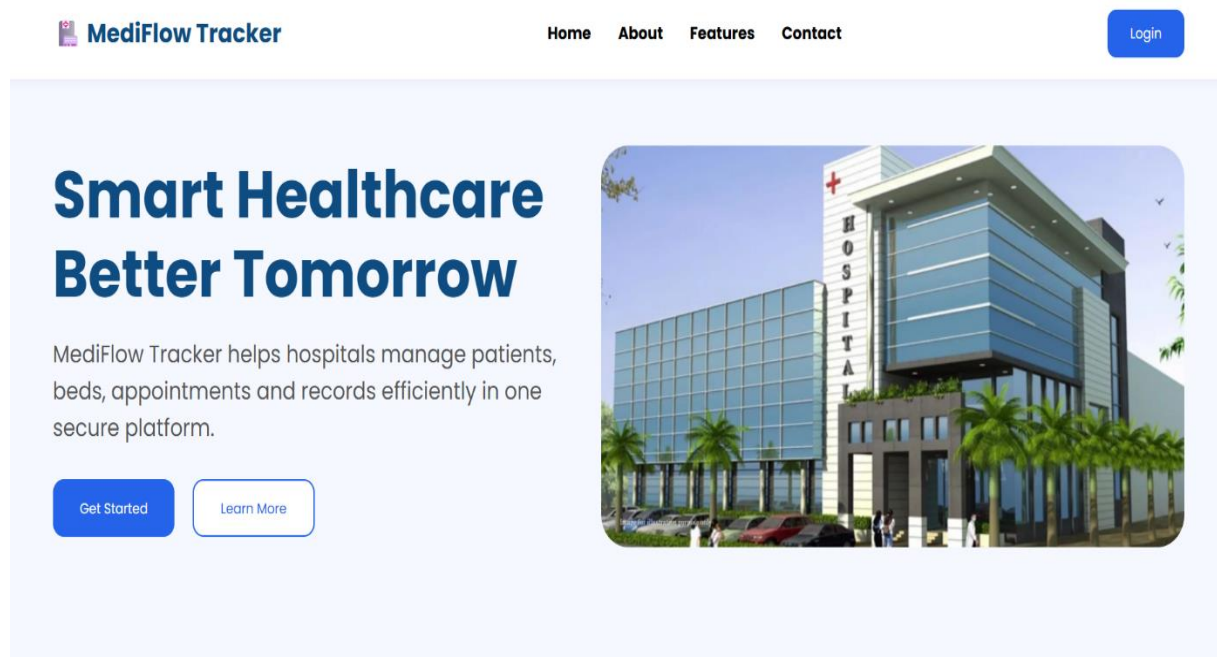
TripAdvisor. (2025). *Tourism and Destination Review Data*. Retrieved from <https://www.tripadvisor.com>

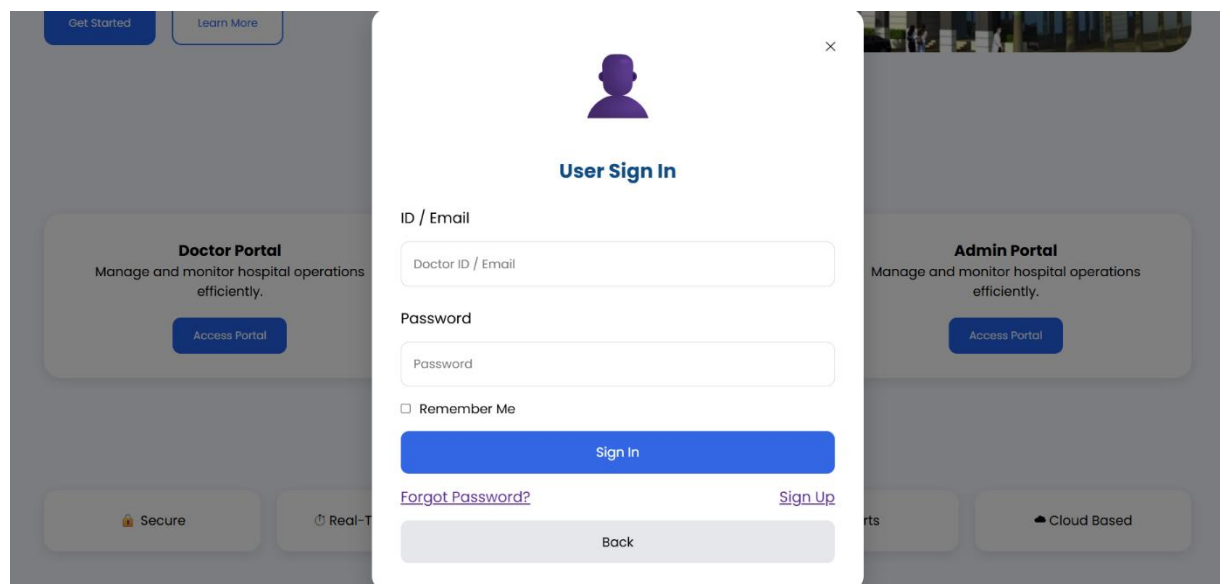
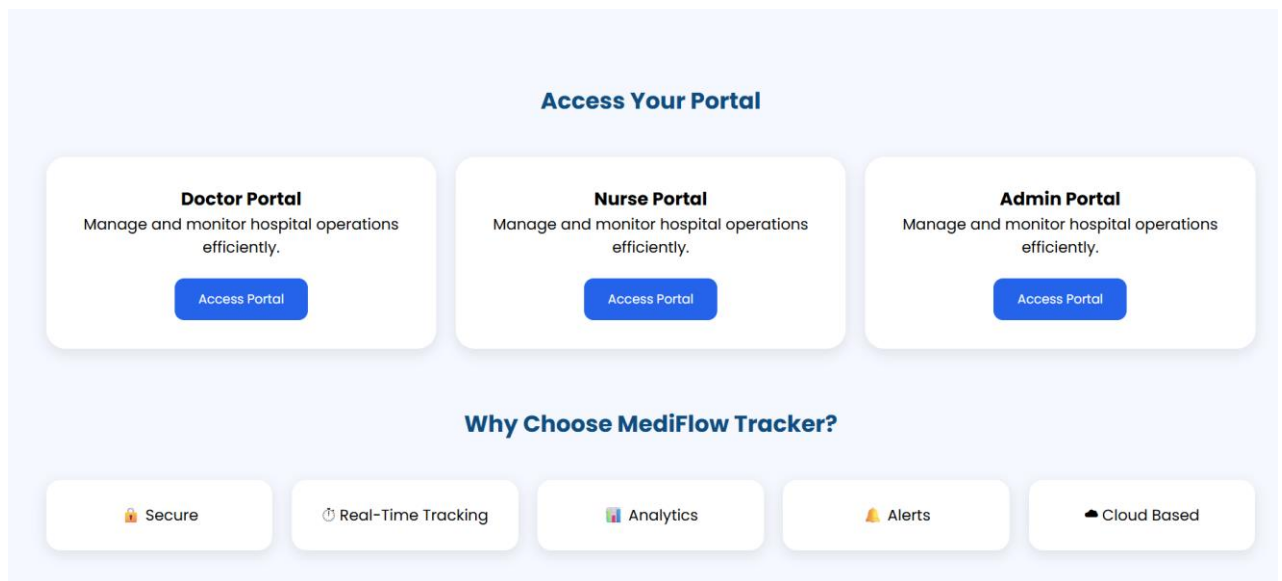
Travel+ Leisure. (2025). *Top Global Destinations and Travel Trends*. Retrieved from <https://www.travelandleisure.com>

Tourism Review. (2025). *Digital Transformation in the Tourism Industry*. Retrieved from <https://www.tourismreview.com>

CHAPTER -14 APPENDICES

➤ Screenshots of the app





Welcome Back, Dr. Harshika

Patient Care & Treatment Monitoring Portal

Patients Under Care

5

[View](#)

Critical Monitoring

2

[View](#)

Treatments In Progress

5

[View](#)

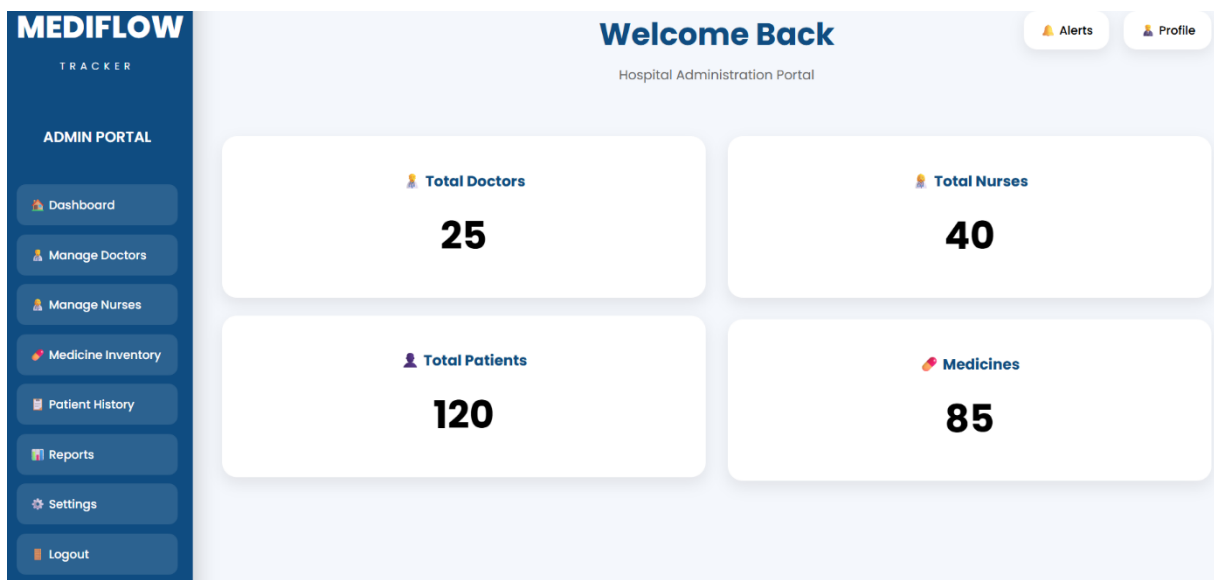
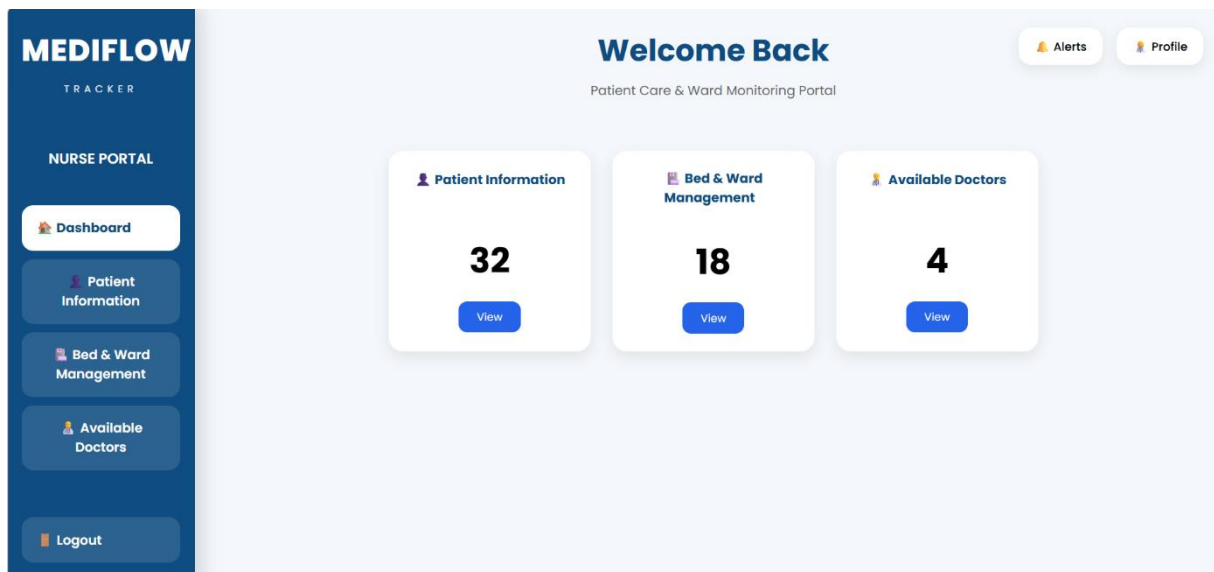
Scheduled Appointments

5

[View](#)

Alerts

Dr. Harshika



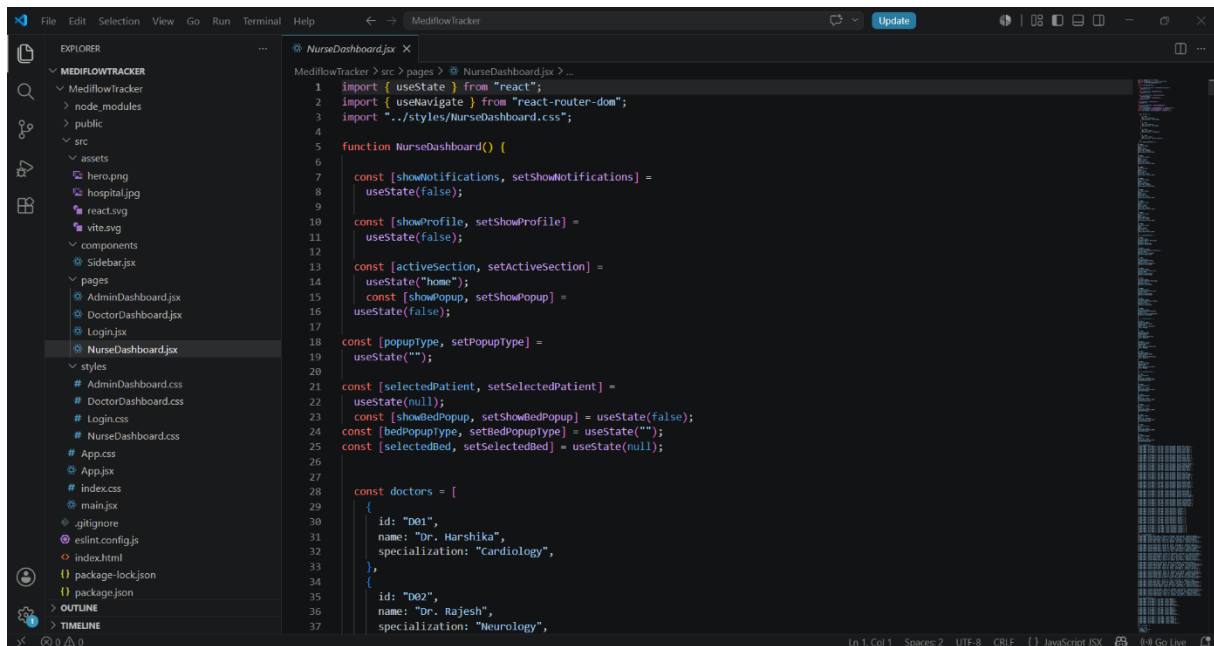
Sample code snippets

```
1 import { Routes, Route } from "react-router-dom";
2
3 import login from "../pages/login";
4 import DoctorDashboard from "../pages/DoctorDashboard";
5 import NurseDashboard from "../pages/NurseDashboard";
6 import AdminDashboard from "../pages/AdminDashboard";
7
8 function App() {
9   return (
10     <Routes>
11       <Route path="/" element=<Login /> />
12       <Route path="/doctor" element=<DoctorDashboard /> />
13       <Route path="/nurse" element=<NurseDashboard /> />
14       <Route path="/admin" element=<AdminDashboard /> />
15     </Routes>
16   );
17 }
18
19 export default App;
```

```
1 import { useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import hospital from "../assets/hospital.jpg";
4 import "../styles/login.css";
5
6 function Home() {
7   const navigate = useNavigate();
8   const [showLogin, setShowLogin] = useState(false);
9   const [role, setRole] = useState("");
10
11   const openLogin = (selectedRole) => {
12     setRole(selectedRole);
13     setShowLogin(true);
14   };
15
16   return (
17     <div>
18       <div>
19         <div>
20           <div>
21             <div>
22               <div>
23                 <div>
24                   <div>
25                     <div>
26                       <div>
27                         <div>
28                           <div>
29                             <div>
30                               <div>
31                                 <div>
32                                   <div>
33                                     <div>
34                                       <div>
35                                         <div>
36                                           <div>
37                                             <div>
```

```
1 import { useState } from "react";
2 import "../styles/DoctorDashboard.css";
3
4 function DoctorDashboard() {
5   const doctorName = "Harshika";
6
7   const [showNotifications, setShowNotifications] =
8     useState(false);
9
10  const [showProfile, setShowProfile] =
11    useState(false);
12
13  const [selectedPatient, setSelectedPatient] =
14    useState(null);
15  const [modalTitle, setModalTitle] =
16    useState("");
17  const [showPatientsModal, setShowPatientsModal] =
18    useState(false);
19
20  const [showCriticalModal, setShowCriticalModal] =
21    useState(false);
22
23  const [showTreatmentsModal, setShowTreatmentsModal] =
24    useState(false);
25
26  const [showAppointmentsModal, setShowAppointmentsModal] =
27    useState(false);
28  const patients = [
29    {
30      id: "P101",
31      name: "Rahul Sharma",
32      age: 45,
33      disease: "Viral Fever",
34      room: "203",
35      bed: "B12",
36      status: "Under Treatment",
37      history: "Visited Jan 2026 for Viral Fever",
38    },
39    {
40      id: "P102",
```

```
1 import { useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import "../styles/AdminDashboard.css";
4
5 function AdminDashboard() {
6   const navigate = useNavigate();
7
8   const [activeSection, setActiveSection] =
9     useState("home");
10
11  const [showNotifications, setShowNotifications] =
12    useState(false);
13
14  const [showProfile, setShowProfile] =
15    useState(false);
16  const [showPatientPopup, setShowPatientPopup] =
17    useState(false);
18
19  const [selectedPatient, setSelectedPatient] =
20    useState(null);
21  const [showReportPopup, setShowReportPopup] =
22    useState(false);
23
24  const [reportType, setReportType] =
25    useState("");
26
27  const doctors = [
28    { id: "D001", name: "Dr. Harshika", department: "Cardiology", experience: "8 Years", status: "Available" },
29    { id: "D002", name: "Dr. Rajesh", department: "Neurology", experience: "10 Years", status: "Available" },
30    { id: "D003", name: "Dr. Priya", department: "Orthopedics", experience: "6 Years", status: "Busy" },
31    { id: "D004", name: "Dr. Arjun", department: "General Medicine", experience: "12 Years", status: "Available" },
32    { id: "D005", name: "Dr. Meera", department: "Pediatrics", experience: "9 Years", status: "Available" },
33    { id: "D006", name: "Dr. Kiran", department: "Dermatology", experience: "7 Years", status: "Busy" },
34    { id: "D007", name: "Dr. Sneha", department: "ENT", experience: "11 Years", status: "Available" },
35    { id: "D008", name: "Dr. Ravi", department: "Cardiology", experience: "5 Years", status: "Available" },
36    { id: "D009", name: "Dr. Anil", department: "Neurology", experience: "14 Years", status: "Busy" },
37  ]
```



➤ Installation/setup instructions

The Hospital MediFlow Tracker Dashboard is developed using **React.js** with **Vite** as the development environment. To set up the project, Node.js (LTS version) must first be installed on the system, as it includes the npm package manager required for installing project dependencies. After downloading the project, open the project folder in Visual Studio Code and execute the npm install command in the terminal. This command installs all the required libraries and packages, including React, React Router DOM, React Icons, and other dependencies specified in the package.json file.

Once the installation is complete, start the application by running the npm run dev command. Vite will launch a local development server and generate a localhost URL (typically <http://localhost:5173>), which can be opened in any modern web browser to access the application. The project consists of separate pages for the Login screen, Doctor Dashboard, Nurse Dashboard, and Admin Dashboard, with external CSS files used for styling and React Router providing smooth navigation between different pages. This setup ensures that the application can be developed, tested, and executed efficiently in a local development environment.

➤ **Geo Tag photos with guide**



➤ **Review forms with guide signatures**